Sesión 5

Clasificación II: KNN, Naive Bayes y SVM

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Más allá de la regresión logística

- La regresión logística es un modelo lineal y probabilístico.
- Muchos problemas requieren enfoques diferentes:
  - Relaciones no lineales.
  - Datos con estructuras específicas (texto, imágenes).
  - Necesidad de interpretabilidad probabilística.
- En esta sesión veremos tres algoritmos clásicos:
  1. **KNN**: basado en similitud geométrica.
  2. **Naive Bayes**: basado en probabilidad condicional.
  3. **SVM**: basado en optimización de márgenes.

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

KNN: Fundamento geométrico

Idea principal

Puntos cercanos en el espacio de características tienden a tener la misma clase.

- No paramétrico, basado en instancias ("perezoso").
- Para un nuevo punto $\mathbf{x}_q$:
  1. Buscar los k vecinos más cercanos en el conjunto de entrenamiento.
  2. Clasificación: voto mayoritario.
  3. Regresión: promedio de los valores.

Métricas de distancia

Euclidiana

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}$$

Minkowski (general)

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^{p} |x_j - y_j|^r \right)^{1/r}$$

Manhattan

$$d(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^{p} |x_j - y_j|$$

Coseno

$$d = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

- Manhattan es más robusta a outliers que Euclidiana.
- Coseno es útil para datos de alta dimensionalidad (texto).

Influencia del parámetro k

k pequeño (ej. $k = 1$)

- Modelo muy flexible.
- Alta varianza (sensible al ruido).
- Riesgo de sobreajuste.

k grande

- Modelo más estable.
- Aumenta el sesgo.
- Puede suavizar en exceso los límites.

Selección de k

Validación cruzada para elegir el k que minimice el error.

Importancia del escalado

¡Crítico en KNN!

KNN se basa en distancias. Si las variables tienen diferentes escalas, las de mayor magnitud dominarán el cálculo.

Estandarización (Z-score)

$$x' = \frac{x - \mu}{\sigma}$$

Normalización (Min-Max)

$$x' = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}$$

- Se prefiere estandarización cuando hay outliers.
- En scikit-learn: `StandardScaler` o `MinMaxScaler`.

Maldición de la dimensionalidad

Problema

Al aumentar el número de dimensiones p, los datos se vuelven dispersos y las distancias entre puntos tienden a ser similares.

- KNN pierde capacidad para encontrar vecinos realmente cercanos.
- Soluciones:
  - Reducción de dimensionalidad (PCA, selección de características).
  - Métricas de distancia alternativas (coseno).
  - Métodos de aproximación (LSH).

Caso de uso: Sistemas de recomendación

Recomendación basada en usuarios

1. Representar usuarios por vectores de valoraciones.
2. Para un usuario activo, encontrar los k usuarios más similares (distancia de coseno o correlación).
3. Recomendar ítems que esos vecinos hayan valorado positivamente y el activo no haya visto.

- Problemas de escalabilidad con millones de usuarios $\rightarrow$ usar índices aproximados (LSH) o clustering previo.
- Cold start: para nuevos usuarios, usar popularidad o contenido.

Implementación en scikit-learn

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

- Parámetros importantes: `n_neighbors`, `metric`, `weights` (unifom o distance).
- Para regresión: `KNeighborsRegressor`.

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Teorema de Bayes

Fórmula

$$P(C_k|\mathbf{x}) = \frac{P(C_k)P(\mathbf{x}|C_k)}{P(\mathbf{x})}$$

donde:

- $P(C_k)$: probabilidad a priori de la clase k.
- $P(\mathbf{x}|C_k)$: verosimilitud (probabilidad de los datos dada la clase).
- $P(C_k|\mathbf{x})$: probabilidad a posteriori.

- Clasificamos eligiendo la clase con mayor probabilidad a posteriori.
- El denominador $P(\mathbf{x})$ es constante y puede ignorarse.

Supuesto de independencia condicional

"Naive" (ingenuo)

Se asume que las características son condicionalmente independientes dada la clase:

$$P(\mathbf{x}|C_k) = \prod_{j=1}^{p} P(x_j|C_k)$$

- Este supuesto rara vez se cumple en la realidad.
- Sin embargo, simplifica enormemente la estimación y el modelo funciona sorprendentemente bien, especialmente en texto.

Variantes de Naive Bayes

GaussianNB

Para características **continuas**:

$$P(x_j|C_k) \sim \mathcal{N}(\mu_{kj}, \sigma_{kj}^2)$$

MultinomialNB

Para **frecuencias** (conteo de palabras):

$$P(\mathbf{x}|C_k) \propto \prod_j \theta_{kj}^{x_j}$$

BernoulliNB

Para **binarias** (presencia/ausencia):

$$P(x_j|C_k) = \theta_{kj}^{x_j}(1-\theta_{kj})^{1-x_j}$$

Suavizado Laplace

Problem

Si una palabra nunca aparece en la clase "spam" en entrenamiento, su probabilidad condicional sería cero, anulando el producto.

Solución: Suavizado Laplace (add-1 smoothing)

$$P(x_j|C_k) = \frac{\text{freq}(x_j, C_k) + \alpha}{\sum_{j'} \text{freq}(x_{j'}, C_k) + \alpha|V|}$$

con $\alpha = 1$ típicamente.

- Evita probabilidades cero.
- Asigna una pequeña probabilidad a palabras no vistas.

Caso de uso: Clasificación de textos (spam)

Filtro de spam

- Características: presencia/frecuencia de palabras.
- Se usa `MultinomialNB` o `BernoulliNB`.
- Independencia condicional: asume que la probabilidad de "viagra" es independiente de "oferta" dado que es spam (falso, pero funciona).
- Rápido, escalable a miles de palabras.

Implementación en scikit-learn

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB

# Para caracter sticas continuas
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# Para frecuencias (texto)
mnb = MultinomialNB(alpha=1.0)
mnb.fit(X_train_counts, y_train)

# Para binarias
bnb = BernoulliNB(alpha=1.0)
bnb.fit(X_train_binary, y_train)
```

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

SVM: Margen máximo

Idea principal

Encontrar el hiperplano que separa las clases con el mayor margen posible.

- Hiperplano: $\mathbf{w}^T\mathbf{x} + b = 0$.
- Decisión: $\text{sign}(\mathbf{w}^T\mathbf{x} + b)$.
- Margen: distancia mínima del hiperplano a los puntos más cercanos $= 2/\|\mathbf{w}\|$.
- Maximizar el margen $\equiv$ minimizar $\frac{1}{2}\|\mathbf{w}\|^2$.

SVM linealmente separable

Problema de optimización primal

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{sujeto a} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \ \forall i$$

- Los puntos con $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ son los **vectores soporte**.
- Sólo ellos determinan el hiperplano; el resto pueden eliminarse.

Margen suave y parámetro C

Para datos no separables linealmente, se introducen variables de holgura ξ_i:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

sujeto a $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$.

- C: controla la penalización por errores.
- C grande $\rightarrow$ margen estrecho, pocos errores (riesgo de sobreajuste).
- C pequeño $\rightarrow$ margen amplio, más errores permitidos (modelo simple).

Truco del kernel

Idea

Proyectar los datos a un espacio de mayor dimensión donde sean separables linealmente, sin calcular explícitamente la transformación $\phi(\mathbf{x})$.

- Se usa una función kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$.

Lineal

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

Polinomial

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$$

RBF

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

Parámetros clave en SVM con kernel RBF

γ (gamma)

Controla la influencia de un solo punto.

- γ grande: solo puntos cercanos influyen (sobreajuste).
- γ pequeño: modelo más suave (subajuste).

C

Penalización por error.

- Interacción con γ: ambos controlan la complejidad.

Búsqueda de hiperparámetros

GridSearchCV o RandomSearchCV con validación cruzada.

Vectores soporte

- Son los puntos que quedan sobre el margen o del lado incorrecto.
- Definen el hiperplano; si se eliminan, el modelo cambia.
- En problemas reales, pueden interpretarse como casos “difíciles” o representativos.

Caso de uso: Reconocimiento facial

Clasificación de caras

- Imágenes convertidas en vectores de píxeles (alta dimensionalidad).
- SVM con kernel RBF puede encontrar límites no lineales.
- Los vectores soporte son las imágenes más representativas o atípicas.

Implementación en scikit-learn

```
from sklearn.svm import SVC

# SVM lineal
svm_lin = SVC(kernel='linear', C=1.0)
svm_lin.fit(X_train, y_train)

# SVM con kernel RBF
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_rbf.fit(X_train, y_train)
```

- Para obtener probabilidades: `probability=True` (más lento).

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Comparación: KNN vs Naive Bayes vs SVM

Característica	**KNN**	**Naive Bayes**	**SVM**
Interpretabilidad	Baja	Alta	Media (lineal) / Baja (RBF)
Entrenamiento	Ninguno (perezoso)	Rápido	Medio (lineal) / Lento (RBF)
Predicción	Lento	Rápido	Rápido
Manejo de no linealidades	Sí (distancia)	No	Sí (con kernel)
Escalado necesario	Sí	No (pero mejora)	Sí
Alta dimensionalidad	Sufre	Robusto	Robusto (con kernel lineal)

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Contexto y datos

20 Newsgroups

- $\approx$ 20,000 documentos en 20 categorías.
- Para simplificar, usamos dos categorías (ej. "medicina" vs "espacio").

Preprocesamiento

1. Tokenización, eliminación de stopwords.
2. Vectorización: CountVectorizer o TF-IDF.
3. Reducción de vocabulario (palabras más frecuentes).
4. División entrenamiento/prueba.

Modelado y comparación

Modelos a probar:

1. KNN con $k = 5$, distancia euclidiana (o coseno).
2. Multinomial Naive Bayes (con suavizado Laplace).
3. SVM lineal.
4. SVM con kernel RBF (con búsqueda de C y γ).

Resultados esperados:

- MultinomialNB: muy bueno, rápido.
- SVM lineal: también excelente, especialmente con TF-IDF.
- SVM RBF: puede mejorar si la frontera no es lineal.
- KNN: puede sufrir por alta dimensionalidad y ser lento.

Análisis e interpretación

- **Naive Bayes**: permite ver las palabras más probables por categoría (inspeccionar θ_{kj}).
- **SVM lineal**: los coeficientes indican importancia de las palabras (para kernel lineal).
- **SVM con RBF**: menos interpretable; se pueden usar técnicas como SHAP o LIME.
- **KNN**: se pueden examinar los vecinos de un documento para entender la decisión.

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Diseño del sistema

- 100,000 pacientes, 50 variables (edad, presión, colesterol, ECG...).
- Objetivo: clasificar enfermedad cardíaca (binario).
- Requisitos: interpretable, alto recall (detectar enfermedad).

Pipeline de preprocesamiento

1. Imputar nulos (mediana).
2. Detectar outliers (IQR) y decidir tratamiento.
3. Estandarizar (Z-score).
4. Selección de características (importancia de árboles, correlación).

Selección de modelo y ajuste

Opciones:

- KNN: k óptimo por validación.
- GaussianNB: rápido, interpretable.
- SVM lineal: interpretable (coeficientes).
- SVM RBF: mejor rendimiento, menos interpretable.

Para maximizar recall:

- Ajustar umbral de decisión (ej. 0.3).
- Usar pesos de clase (class_weight='balanced').
- Sobremuestreo (SMOTE) con cuidado.

Evaluación y monitoreo

- **Métricas:** matriz de confusión, recall, precisión, F1, AUC-ROC, curva Precision-Recall.
- Priorizar recall, pero controlar precisión para evitar falsos positivos.
- **Interpretabilidad:** usar SHAP para explicar predicciones de SVM RBF.
- **Monitoreo en producción:** detectar data drift con PSI, KS, y reentrenar si es necesario.

Contenido

1. Introducción
2. K-Vecinos más cercanos (KNN)
3. Naive Bayes
4. Máquinas de Vectores de Soporte (SVM)
5. Comparación
6. Caso de estudio: Clasificación de noticias
7. Caso avanzado: Diagnóstico médico
8. Conclusión

Lo que hemos aprendido

Conceptos clave

- **KNN**: clasificador basado en instancias, sensible a escala y dimensionalidad.
- **Naive Bayes**: modelo probabilístico con supuesto de independencia, muy efectivo en texto.
- **SVM**: maximiza el margen, con truco del kernel para no linealidades.
- Comparación de fortalezas y debilidades según el problema.
- Aplicaciones a recomendación, spam, reconocimiento facial, diagnóstico médico.

Conexión con siguientes sesiones

Estos conceptos se extienden a métodos ensemble (Random Forest, Gradient Boosting) y redes neuronales.

¡Gracias!